

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43
COURSE OUTCOME COVERED

A.LAKSHMI(192373066)

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis :Consider the following DOM tree representation:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.

4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods: **Method URL Example Data Visibility Security Level** GET /login?user=admin&pass=1234 Visible in URL Low POST /login Hidden in body Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I A.LAKSHMI(192373066) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

1. HTTP Response Analysis

Given HTTP Response Header

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

1. Identify any incorrect or invalid fields in the header.

- The **Content-Length** field is invalid because it has a negative value (-150).
- All other fields (HTTP/1.1, 200 OK, Date, Content-Type, and Connection) are valid.

2. Analyze the issue with Content-Length and explain its impact.

- **Content-Length** specifies the size of the response body in bytes.
- A negative value is not allowed.
- Because of this:
 - The browser cannot determine the correct response size.
 - The response may fail to load correctly.
 - The browser may ignore the response or display an error.

3. Determine whether the status code matches the response correctness.

- The status code **200 OK** means the request was processed successfully.
- However, since the response contains an invalid **Content-Length**, the response is technically incorrect.
- Therefore, the status code does **not fully match** the response because the header contains an error.

4. Suggest corrections for the identified errors.

Correct the **Content-Length** to the actual file size.

Example:

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive

2. DOM Structure Analysis

Given DOM Structure

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

1. Identify missing or improperly closed elements.

- The `<p>` tag is missing its closing tag `</p>`.
- The `<div>` is closed before the paragraph is explicitly closed.

2. Analyze how the browser will interpret this structure.

- Modern browsers automatically close unclosed tags.
- The browser will treat it as:

```
<div>
  <p>HelloWorld</p>
</div>
```

3. Determine the impact on rendering and layout.

- Most browsers will still display the text correctly.
- However:
 - The HTML is invalid.
 - Different browsers may interpret it differently.
 - It may create unexpected layout or styling issues.

4. Suggest corrections to make the DOM valid.

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld</p>
</div></body></html>
```

3. GET vs POST in Login System

1. Interpret the differences in data transmission between GET and POST.

GET

- Sends data through the URL.
- Data is visible in the browser address bar.
- Limited amount of data can be sent.
- Used for retrieving information.

POST

- Sends data in the request body.
- Data is not visible in the URL.
- Can send large amounts of data.
- Used for submitting sensitive information.

2. Analyze which method is more suitable for login and why.

POST is the better choice because:

- Username and password are hidden from the URL.
- Credentials are not stored in browser history.
- Provides better security when used with HTTPS.

3. Identify security risks in using GET for authentication.

Using GET may:

- Expose usernames and passwords in the URL.
- Save credentials in browser history.
- Store sensitive information in server logs.
- Allow passwords to be shared accidentally through copied URLs.

4. Suggest best practices for secure login implementation.

- Use **POST** for login forms.
- Use **HTTPS** to encrypt communication.
- Hash passwords before storing them.
- Validate user input.
- Use secure session management.
- Implement multi-factor authentication (MFA) if possible.

4. HTML Code Inspection

Given HTML Code

```
<!DOCTYPE html>

<html>

<head>

<title>Sample</title>

</head>

<body>

<h1>Welcome

<p>This is a paragraph



</body>

</html>
```

1. Identify missing closing tags.

Missing tags:

- `</h1>`
- `</p>`

2. Analyze issues with the `<img>` tag.

- The `<img>` tag is valid because it is a **void element** and does **not require a closing tag**.
- However, it should include an **alt** attribute for accessibility.

Example:

```

```

3. Determine how browsers will handle these errors.

Browsers automatically close the missing tags and attempt to display the page correctly.
However:

- HTML becomes invalid.
- Styling and page layout may behave unexpectedly.

4. Corrected HTML Code

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Sample</title>
```

```
</head>
```

```
<body>
```

```
<h1>Welcome</h1>
```

```
<p>This is a paragraph</p>
```

```

```

```
</body>
```

```
</html>
```


5. Browser–Server Communication Flow

Communication Steps

- User enters URL
- DNS lookup
- HTTP request
- Server processing
- HTTP response
- Browser rendering

1. Interpret each step in the communication flow.

Step 1 – User enters URL

The user types a website address into the browser.

Step 2 – DNS Lookup

DNS converts the domain name into the server's IP address.

Step 3 – HTTP Request

The browser sends an HTTP request to the web server.

Step 4 – Server Processing

The server processes the request and retrieves the required data.

Step 5 – HTTP Response

The server sends the requested web page and resources back to the browser.

Step 6 – Browser Rendering

The browser interprets HTML, CSS, and JavaScript to display the webpage.

2. Identify where delays can occur.

Delays may occur during:

- DNS lookup.
- Slow internet connection.
- Network latency.
- Server processing.
- Downloading large images, videos, or scripts.

3. Analyze the role of DNS in the process.

- DNS translates the website name into an IP address.
- Without DNS, browsers would not know which server to contact.
- It acts like the Internet's phonebook.

4. Suggest optimizations to improve performance.

- Use DNS caching.
- Enable browser caching.
- Compress files using Gzip or Brotli.
- Optimize images.
- Minify CSS and JavaScript.
- Use a Content Delivery Network (CDN).
- Reduce unnecessary HTTP requests.

Summary Table

Question	Key Answer
1. HTTP Response Analysis	Correct invalid <code>Content-Length</code> ; use a positive value matching the response size.
2. DOM Structure Analysis	Close the missing <code></p></code> tag to create valid HTML.
3. GET vs POST	Use POST with HTTPS for secure login.
4. HTML Inspection	Add missing <code></h1></code> and <code></p></code> tags; include an <code>alt</code> attribute for <code></code> .
5. Browser–Server Communication	Flow: URL → DNS → HTTP Request → Server → HTTP Response → Browser Rendering. Optimize using caching, compression, and CDN.